

PERSONAL EXPENSE TRACKER

<https://github.com/arroart/OOPFinalExpenseTracker>

Group:

Lia Jang (lj1353)

Arnav Arora (aa10027)

Joseph Cheng (jxc2008)

Date of Submission: **05/02/2026**

Table Of Work

Work	Arnav Arora	Lia Jang	Joseph Cheng
Project Description	X	X	
Use Case Diagrams and description			X
Sequence Diagrams		X	
Class diagram(s)	X		
Implementation	X	X	X
Expense Class	X		
Category Class	X		
Expense Manager			X
Expense File Save			X
GUI- All the User Interface		X	
Conclusion	X		

System Analysis

General Description, Goals, and Benefits

- ❖ This project is a Personal Expense Tracker.
- ❖ It helps users record and track their daily spending.
- ❖ Users can enter the amount, category, date, and description for each expense.
- ❖ The system stores all expenses and allows users to view them anytime.
- ❖ Users can:
 - Add new expenses
 - Edit existing expenses
 - Delete expenses
 - View all expenses
 - Filter expenses by category or date
- ❖ The system also provides a monthly summary:
 - Shows total spending
 - Shows spending by category
- ❖ The main goal is to help users understand where their money goes.
- ❖ This helps users make better financial decisions.
- ❖ The system is simple and easy to use, especially for students.
- ❖ Main classes used:
 - Expense (stores expense data)
 - Category (stores type of expense)
 - ExpenseManager (handles logic and data)
 - UserInterface (handles GUI and user interaction)

Special Requirements

Performance

- ❖ The system should run fast for normal use.
- ❖ It should handle adding, deleting, and filtering without delay.
- ❖ It is designed for personal use, so large-scale performance is not required.

Interfaces

- ❖ The system uses a GUI built with Java Swing.
- ❖ Users interact using:
 - Text fields
 - Buttons
 - Dropdown (for category)
 - Text area (to display results)
- ❖ The interface is simple and easy to understand.

Constraints

- ❖ The system is built using Java.
- ❖ It uses basic data structures like lists.
- ❖ Data is stored in file, and stays there even after program is closed

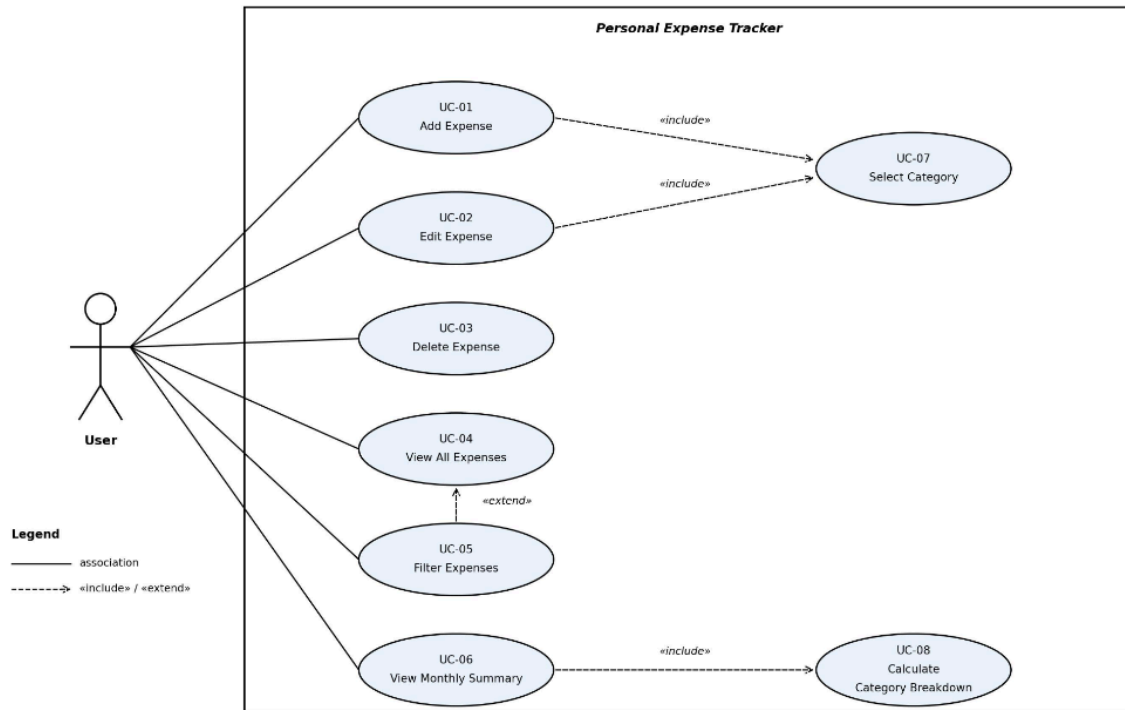
Reliability

- ❖ The system should handle user input properly.
- ❖ It should avoid crashes for basic use.
- ❖ Simple validation can be added (e.g., no negative amounts).
- ❖ Since it runs locally, it does not depend on internet connection.

System Design

Use-Case Diagram

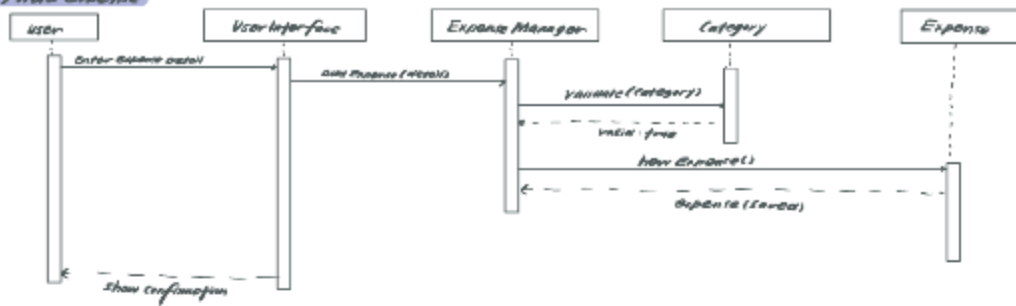
Personal Expense Tracker — Use-Case Diagram



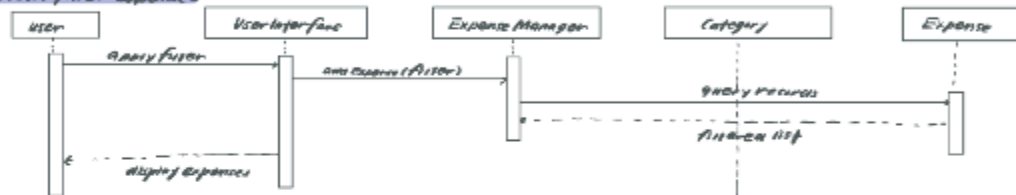
ID	Name	Actor	Description	Preconditions	Main Flow	Postconditions
UC-01	Add Expense	User	Record a new expense with amount, date, description, category.	Application is running; categories list available.	1. User opens 'Add Expense' form. 2. User selects a category (UC-07). 3. User enters amount, date, description. 4. System validates input (non-null, amount ≥ 0). 5. System stores the expense.	Expense is saved and visible in the expense list.
UC-02	Edit Expense	User	Modify the fields of an existing expense.	At least one expense exists; user has selected one.	1. User selects expense and chooses 'Edit'. 2. Form pre-fills with current values. 3. User changes fields; may reselect category (UC-07). 4. System validates and replaces the stored expense.	Selected expense is updated in place.
UC-03	Delete Expense	User	Remove an existing expense from the tracker.	At least one expense exists; user has selected one.	1. User selects expense and chooses 'Delete'. 2. System removes the expense from storage. 3. List view refreshes.	Expense is no longer present.
UC-04	View All Expenses	User	Display every recorded expense in a list.	Application is running.	1. User opens the main expenses view. 2. System fetches all expenses. 3. System renders them in chronological / insertion order.	User sees the full expense list (empty list if none).
UC-05	Filter Expenses	User	Narrow the expense list by category or date. Extends UC-04.	User is on the 'View All Expenses' screen.	1. User selects a filter (category or date). 2. System computes the matching subset. 3. List view shows only the filtered results.	Only matching expenses are displayed; clearing the filter restores UC-04 view.
UC-06	View Monthly Summary	User	See total spending grouped by category for a given month/year.	Application is running; one or more expenses exist.	1. User selects month and year. 2. System runs UC-08 to compute totals per category. 3. System displays the breakdown.	Per-category totals for the selected month are shown.
UC-07	Select Category	User	Choose a category for an expense being added or edited. Included by UC-01 and UC-02.	Add or Edit Expense flow is in progress.	1. System presents available categories. 2. User picks one. 3. System validates the choice and returns control to the calling use case.	A valid category is attached to the expense in flight.
UC-08	Calculate Category Breakdown	System	Aggregate expenses by category over a month/year. Included by UC-06.	Triggered from UC-06 with a month and year.	1. System filters expenses to the given month/year. 2. System groups them by category name. 3. System sums amounts per category and returns the map.	A category $\rightarrow$ total-spend map is returned to UC-06.

Sequence Diagram

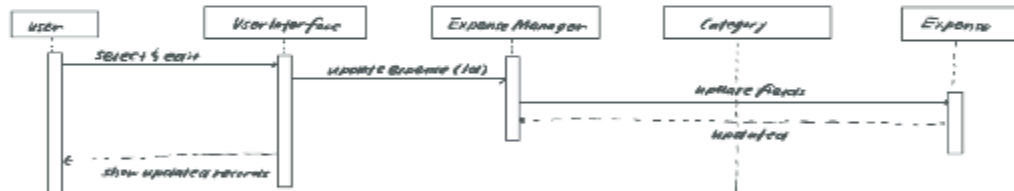
1) Add Expense



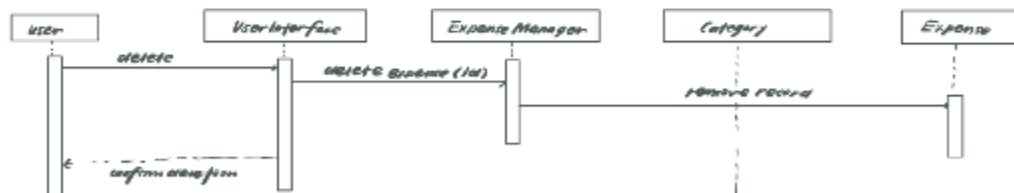
2) View/Filter Expenses



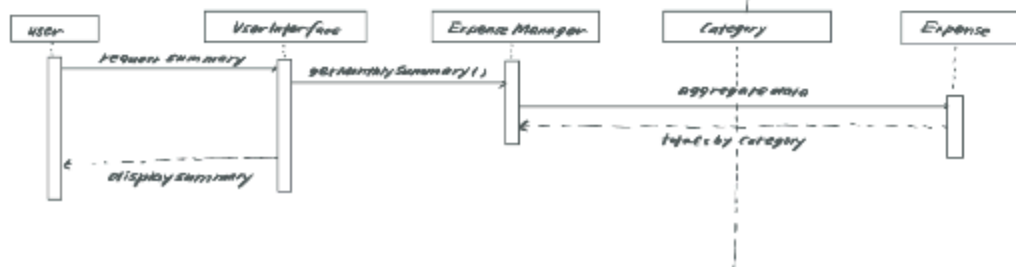
3) Edit Expense



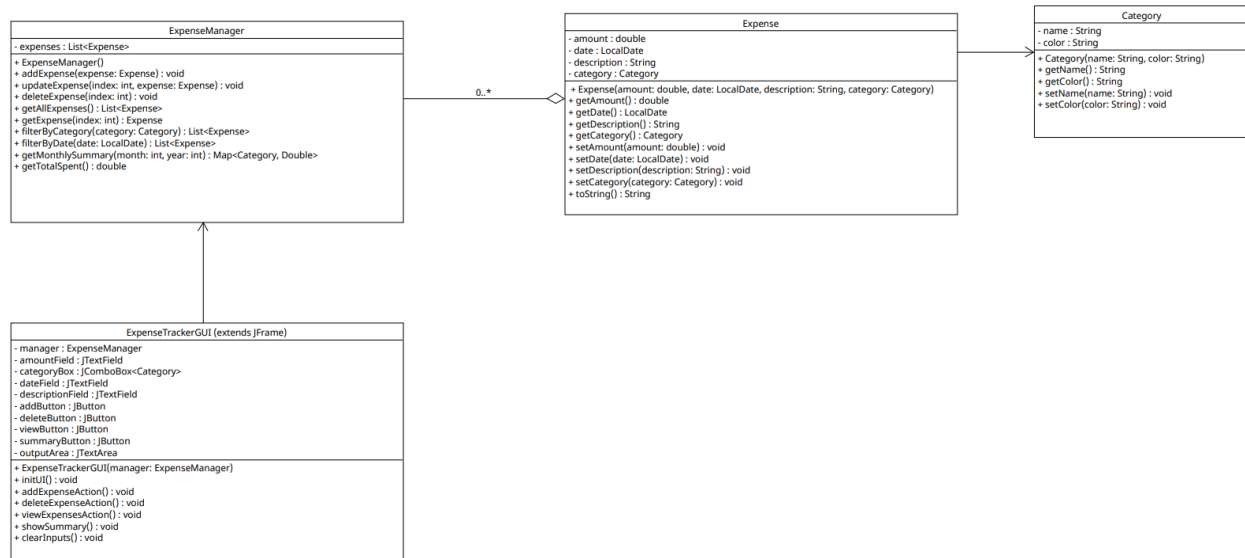
4) Delete Expense



5) Generate Monthly Summary



Class Diagram



Conclusion

During the development process, one of the main difficulties was designing the system structure correctly. Deciding how classes should interact, especially between **Expense**, **Category**, and **ExpenseManager**, required careful thinking. Another challenge was implementing the graphical user interface using Java Swing, as it required understanding how different components work together and how to handle user actions properly.

Another difficulty was just handling GitHub, we actually had a very weird experience with merging and had to remake the repo one time.

Despite these challenges, the project helped improve understanding of object-oriented programming concepts such as class design, relationships, and separation of responsibilities. It also provided practical experience in building a working application with a user interface.

A feature that we interpreted as needed was saving data locally, so the user can see their budget saved when they open the app again.

Additional features such as charts or graphs could be added to better visualize spending patterns. The interface could also be improved to make it more interactive and visually appealing.

Overall, this project was a valuable learning experience and demonstrates how software can be used to solve everyday problems in a simple and effective way.